Alw3lab

Preliminary Audit Report

Mon Jan 26 2026

contact@bitslab.xyz https://twitter.com/scalebit_

ScaleBit

AIw3lab Preliminary Audit Report

1 Executive Summary

1.1 Project Information

Description	A Solidity smart contract project for futures margin pool management with role-based access control, invest item management, configurable commission rates, and per-deposit time locks.
Type	Staking
Auditors	jolyon, s3cunda
Timeline	Thu Jan 22 2026 - Thu Jan 22 2026
Languages	Solidity
Platform	Ethereum
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/aiw3lab/aiw3-user-vault-management
Commits	c5f7dee55fa6986191ab9b0d5307d2c5ff9f2dc3

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
FMPC	contracts/FuturesMarginPoolClassics.sol	aa9a9bddbeafac22abf37be046b01e46a3955939

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	5	0	0
Centralization	1	0	0
Critical	0	0	0
Major	1	0	0
Medium	2	0	0
Minor	0	0	0
Informational	1	0	0
Discussion	0	0	0

1.4 ScaleBit Audit Breakdown

ScaleBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow
- Number of rounding errors
- Unchecked External Call
- Unchecked CALL Return Values
- Functionality Checks
- Reentrancy
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic issues
- Gas usage
- Fallback function usage
- tx.origin authentication
- Replay attacks
- Coding style issues

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by AIw3lab to identify any potential issues and vulnerabilities in the source code of the AIw3lab smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 5 issues of varying severity, listed below.

ID	Title	Severity	Status
FMP-1	Update to the `marginCoinAddress` May Lead to User Asset Locking	Medium	Pending
FMP-2	Updates to the `InvestItem.commissionBps` Parameter May Cause Inconsistent Fee Calculations	Medium	Pending
FMP-3	Deposit Hash Front-Running Attack	Major	Pending
FMP-4	Missing Event Emission in `cancelAdminTransfer()` Function	Informational	Pending
FMP-5	Centralization Risks	Centralization	Pending

3 Participant Process

Here are the relevant actors with their respective abilities within the AIw3lab Smart Contract :

Admin

- `withdraw` - Processes a withdrawal for a user from a specific deposit.
- `withdrawWithItem` - Processes a withdrawal for a user using invest item's commission rate from deposit.
- `withdrawAdminFun` - Transfers funds from the pool to vaults (admin or operator only).
- `pause` - Pauses all deposit and withdrawal operations (admin only).
- `unpause` - Unpauses all operations (admin only).
- `modifyMarginAddress` - Updates the margin token address (admin only).
- `modifyWithdrawAdmin` - Updates the withdrawal admin address (admin only).
- `modifyVaultsAddress` - Updates the vaults address (admin only).
- `modifyFeeAddress` - Updates the fee address (admin only).
- `transferAdmin` - Initiates admin transfer to a new address (two-step process).
- `acceptAdmin` - Completes the admin transfer (must be called by pending admin).
- `cancelAdminTransfer` - Cancels a pending admin transfer (admin only).
- `addOperator` - Adds an operator (admin only).
- `removeOperator` - Removes an operator (admin only).
- `createInvestItem` - Creates a new invest item (admin or operator only).
- `setInvestItemStatus` - Sets the active status of an invest item (admin or operator only).

- `setInvestItemCommission` - Sets the commission rate of an invest item (admin or operator only).
- `setInvestItemLockDuration` - Sets the minimum lock duration of an invest item (admin or operator only).

User

- `deposit` - Deposits margin tokens into the pool with time lock.

4 Findings

FMP-1 Update to the `marginCoinAddress` May Lead to User Asset Locking

Severity: Medium

Status: Pending

Code Location:

contracts/FuturesMarginPoolClassics.sol

Descriptions:

The `FuturesMarginPoolClassics` contract allows users to deposit ERC20 tokens from the address specified as `marginCoinAddress` via the `deposit` function. However, during the deposit process, the current `marginCoinAddress` is not recorded, and the admin can update `marginCoinAddress` at any time through the `modifyMarginAddress` function. This creates an issue: if a user deposits `tokenA` while the admin changes the address to `tokenB` during the process, the user will not receive `tokenA` upon withdrawal. As a result, the user's assets may become locked, and there is even a risk of asset loss due to mismatched parameters such as price or decimal places.

Suggestion:

It is recommended to record the `marginCoinAddress` at the time of deposit. During withdrawal, the contract should verify the `marginCoinAddress` recorded at the time of deposit to prevent inconsistencies caused by updates.

FMP-2 Updates to the `InvestItem.commissionBps` Parameter May Cause Inconsistent Fee Calculations

Severity: Medium

Status: Pending

Code Location:

contracts/FuturesMarginPoolClassics.sol

Descriptions:

The `FuturesMarginPoolClassics.withdrawWithItem` function can be called by the admin, and it calculates the commission fee based on `InvestItem.commissionBps`. During a user's deposit process, the current `InvestItem.commissionBps` is not recorded, and the admin can update `InvestItem.commissionBps` at any time via the `setInvestItemCommission` function. This creates an issue: a user may deposit when `InvestItem.commissionBps` is value A, but if the admin changes it to value B during this period, the fee rate applied at withdrawal will no longer match the rate the user originally agreed to at deposit. This could result in an inconsistent and potentially unfair commission calculation.

Suggestion:

It is recommended to record the `commissionBps` value in effect at the time of each deposit. During withdrawal, the contract should reference the `commissionBps` value recorded at deposit time. This ensures fee consistency and prevents discrepancies caused by subsequent updates to the parameter.

FMP-3 Deposit Hash Front-Running Attack

Severity: Major

Status: Pending

Code Location:

contracts/FuturesMarginPoolClassics.sol#172

Descriptions:

contracts/FuturesMarginPoolClassics.sol

```
function deposit(
    uint256 depositAmount,
    uint256 investItemId,
    uint256 lockDuration,
    bytes32 depositHash
) public nonReentrant whenNotPaused {
    require(depositAmount > 0, "FuturesMarginPool/ZERO_AMOUNT");
    require(!depositFlag[depositHash], "FuturesMarginPool/DUPLICATE_DEPOSIT_HASH");
    // ... omitted business logic ...
    depositFlag[depositHash] = true;
    // ... omitted business logic ...
}
```

The `deposit()` function accepts a user-provided `depositHash` parameter, which is used as a unique identifier to prevent duplicate deposits. Since this hash is visible in the mempool before transaction confirmation, malicious actors can monitor pending deposit transactions and front-run them by submitting their own transaction with the same depositHash but higher gas price.

When the attacker's transaction is mined first, the `depositFlag[depositHash]` is set to true, causing the victim's original transaction to revert with `DUPLICATE_DEPOSIT_HASH`. This a Denial-of-Service (DoS) attack vector that blocks legitimate user deposits. The attack is

economically viable as it incurs minimal cost to the attacker: merely the network gas fees plus a negligible deposit amount (e.g., 1 unit of marginCoin).

Suggestion:

It is recommended to introduce a minimum deposit amount to increase the economic cost of performing such attacks, or generate the depositHash on-chain within the deposit() function instead of using a user-provided parameter directly. The hash calculation should incorporate msg.sender along with other transaction-specific parameters.

FMP-4 Missing Event Emission in `cancelAdminTransfer()` Function

Severity: Informational

Status: Pending

Code Location:

contracts/FuturesMarginPoolClassics.sol#435

Descriptions:

contracts/FuturesMarginPoolClassics.sol

```
function cancelAdminTransfer() public onlyAdmin {
    pendingAdmin = address(0);
}
```

The `cancelAdminTransfer()` function modifies the pendingAdmin state variable but does not emit an event to log this action.

Suggestion:

It is recommended to define and emit an event when the admin transfer is cancelled.

FMP-5 Centralization Risks

Severity: Centralization

Status: Pending

Code Location:

contracts/FuturesMarginPoolClassics.sol

Descriptions:

Several centralization risks were identified in `FuturesMarginPoolClassics` contract:

1. The administrator can withdraw all tokens from the contract's `marginCoinAddress` via the `withdrawAdminFun` function.
2. User-deposited assets can only be withdrawn through the `withdraw` or `withdrawWithItem` functions, both of which require administrator privileges.
3. The administrator can pause or unpause the contract.
4. The administrator controls key deposit parameters, including the token address, fee rates, lock-up periods, and other related settings.
5. The administrator manages contract permissions and has the authority to arbitrarily set or modify administrator and operator addresses.

Suggestion:

It is recommended to implement a multi-signature mechanism combined with a timelock for administrator addresses. This approach helps mitigate the risk of user or protocol asset loss due to private key compromise.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.